R for Bioinformatics Assignment

OlamideOkunola_S3559438

2026-05-18

DECLARATION OF GENERATIVE AI USE: • I DID NOT use Generative AI technology in the development, writing, or editing of this assignment

library(ggplot2) library(limma) library(VennDiagram) library(grid) knitr::opts_knit$set(root.dir = "~/Downloads/R for Bioinformatics assessment")

Q1 (20 marks) What is R and why is it widely used in bioinformatics? Discuss its advantages and disadvantages.

•List and describe some basic data structures in R (e.g., vectors, matrices, data frames). •Explain with examples the differences between data types in R, such as numeric, character, logical, and factor.

What is R?

R is a free, open-source statistical computing environment and programming language. It was first created in the early 1990s at the University of Auckland by Ross Ihaka and Robert Gentleman (1996). It has since developed into one of the most used tools for scientific research and data analysis (R Core Team, 2024).

R is more widely used in bioinformatics since it was designed with statistics in mind from the beginning. R simplifies statistical testing, which is nearly always necessary for analysing biological data, including gene expression values, sequence counts, and clinical assessments. Built on top of R, the Bioconductor project offers hundreds of specialised packages for proteomics, transcriptomics, genomics, and related topics.

Advantages of R in Bioinformatics

Statistical depth: Compared to general-purpose languages, R's vast array of built-in statistical functions and packages makes tasks like survival modelling and differential expression analysis much easier (Ihaka and Gentleman, 1996). **Bioconductor**: This repository adds thousands of peer-reviewed, actively maintained bioinformatics-specific tools, such as limma, DESeq2, and edgeR . **Data visualisation**: With comparatively little code, packages like ggplot2 generate figures of publishing quality (Gentleman et al., 2004). **Vibrant community**: Through sites like the Bioconductor support page and Stack Overflow, there is a thriving community with an abundance of tutorials, documentation, and forum support. **Code, output, and narrative prose can all be merged into a single reproducible document using R Markdown.

Disadvantages of R

Storage management: By default, R puts all data into RAM, which could be problematic for really large datasets such as whole-genome sequencing data (Gillespie and Lovelace, 2017). **Speed**: Although packages like Rcpp can help, R may be slower than compiled languages like C++ or Python for computationally demanding loops (Eddelbuettel and François, 2011). **Learning curve**: Beginners may find the syntax inconsistent and the range of methods (basic R, tidyverse, data.table) perplexing. **By default, Base R is single-threaded, meaning it does not automatically use multiple CPU cores. —

Basic Data Structures in R

For effective data organization and storage, R offers a number of built-in data structures. In bioinformatics, vectors, matrices, and data frames are the three most often used structures (Wickham and Grolemund, 2017).

Vectors

The simplest basic data structure in R is a vector, which holds a series of identically typed values.

```
# Below is a simple numeric vector of expression values:
expression_values <- c(4.2, 3.9, 7.3, 2.6, 6.8)
expression_values
```

```
## [1] 4.2 3.9 7.3 2.6 6.8
```

```
# Below is a character vector of gene names:
gene_names <- c("BRCA1", "HER2", "MYC", "EGFR", "KRAS")
gene_names
```

```
## [1] "BRCA1" "HER2"  "MYC"   "EGFR"  "KRAS"
```

```
## to access the elements in the gene_names
gene_names[3]
```

```
## [1] "MYC"
```

```
expression_values[expression_values > 5]
```

```
## [1] 7.3 6.8
```

Matrices

A two-dimensional arrangement in which every element has the same kind is called a matrix. A well-known example in bioinformatics is a gene expression matrix with samples as columns and genes as rows (Gentleman et al., 2004).

```
# Creating a small expression matrix as illustration: 3 genes x 4 samples
exp_matrix <- matrix(
  c(4.1, 5.2, 2.9, 6.0,
    7.2, 3.1, 8.4, 5.6,
    1.6, 4.9, 3.3, 7.1),
  nrow = 3,
  ncol = 4,
  byrow = TRUE
)
rownames(exp_matrix) <- c("Gene_A", "Gene_B", "Gene_C")
colnames(exp_matrix) <- c("Sample1", "Sample2", "Sample3", "Sample4")
exp_matrix
```

```
##        Sample1 Sample2 Sample3 Sample4
## Gene_A     4.1     5.2     2.9     6.0
## Gene_B     7.2     3.1     8.4     5.6
## Gene_C     1.6     4.9     3.3     7.1
```

```
# To access row 2, since it is [r, c]:
exp_matrix[2, ]
```

```
## Sample1 Sample2 Sample3 Sample4
##     7.2     3.1     8.4     5.6
```

```
# proceeding further to calculate mean expression per gene (across the samples abo
ve)
rowMeans(exp_matrix)
```

```
## Gene_A Gene_B Gene_C
##  4.550  6.075  4.225
```

Data Frames

The widely used R structure for tabular data is a data frame. It is perfect for storing sample metadata (e.g., patient age, disease status, gene name) since columns can include a variety of data types, unlike a matrix.

```
patient_info <- data.frame(
  SampleID    = c("S001", "S002", "S003", "S004"),
  Age         = c(42, 57, 36, 63),
  Disease     = c("AML", "Control", "AML", "Control"),
  stringsAsFactors = FALSE
)
patient_info
```

```
##   SampleID Age Disease
## 1     S001  42     AML
## 2     S002  57 Control
## 3     S003  36     AML
## 4     S004  63 Control
```

```
# Selecting AML patients only:
patient_info[patient_info$Disease == "AML", ]
```

```
##   SampleID Age Disease
## 1     S001  42     AML
## 3     S003  36     AML
```

```
# next line is to add a column for age bracket
patient_info$AgeBracket <- ifelse(patient_info$Age > 50, "Over50", "Under50")
patient_info
```

```
##   SampleID Age Disease AgeBracket
## 1     S001  42     AML    Under50
## 2     S002  57 Control     Over50
## 3     S003  36     AML    Under50
## 4     S004  63 Control     Over50
```

Data Types in R

Since each type affects how values are saved, altered, and interpreted during analysis, understanding data types is essential to programming in R (Wickham and Grolemund, 2017).

Numeric

In R, numerical values are the default for numbers and represent real numbers (R Core Team, 2024). For storing continuous measurements, it is the most often used data type in bioinformatics. Both a chromosome length and a TPM expression value are stored as numerical numbers in the example below.

```
tpm_value <- 12.5    # TPM (transcripts per million)
chromosome_length <- 2.49*10^8  # chromosome 1 length in bp
class(tpm_value)
```

```
## [1] "numeric"
```

```
is.numeric(tpm_value)
```

```
## [1] TRUE
```

Character

In bioinformatics, character data is often used for biological sequences (sequence strings), gene identifiers, and sample IDs. A short DNA sequence fragment and an Ensembl gene ID are both stored as character objects in the example below.

```
gene_id <- "ENSG00000012048"
sequence_fragment <- "ATGCGTACGTTAG"
class(gene_id)
```

```
## [1] "character"
```

```
nchar(sequence_fragment)   # in order to count characters
```

```
## [1] 13
```

Logical

The two logical values are TRUE and FALSE. They frequently show up when verifying conditions or filtering data (R Core Team, 2024). They are very helpful in bioinformatics when it comes to filtering datasets according to thresholds like significance cutoffs.

```
# i can set a flag, thereby indicating significance
is_significant <- TRUE
p_value <- 0.03   # proceeding to define the p-value
passes_threshold <- p_value < 0.05
passes_threshold    # to view the result (TRUE or FALSE)
```

```
## [1] TRUE
```

```
class(passes_threshold)     # check the data type of the result
```

```
## [1] "logical"
```

```
# Using logical condition in subsetting or filtering values
scores <- c(0.01, 0.23, 0.001, 0.6, 0.049)
scores[scores < 0.05]   # Keep only scores that are below 0.05 i.e. significant sc
ores
```

```
## [1] 0.010 0.001 0.049
```

Factor

Factors are used to record the underlying values of categorical variables as numbers with labels. In addition, for grouping samples in statistical models, they are especially helpful (Chambers, 2008).

```
# Create a factor to represent the disease group
disease_group <- factor(c("AML", "Control", "AML", "AML", "Control"))
# Quick checks:
# View the factor values
disease_group
```

```
## [1] AML     Control AML     AML     Control
## Levels: AML Control
```

```
levels(disease_group)  # to Check the levels
```

```
## [1] "AML"     "Control"
```

```
table(disease_group)  # to count how many are in each group
```

```
## disease_group
##     AML Control
##       3       2
```

```
# to create ordered factor for the tumour_severity levels or grades will be:
tumour_severity <- factor(c("Low", "High", "Medium", "Low", "High"),
                  levels = c("Low", "Medium", "High"),
                  ordered = TRUE)
tumour_severity
```

```
## [1] Low    High   Medium Low    High
## Levels: Low < Medium < High
```

```
tumour_severity[1] < tumour_severity[2]   # TRUE, because ordered is defined
```

```
## [1] TRUE
```

#Q2 (20 marks) A biochemist is analysing the hydrophobicity of protein sequences to identify regions enriched in hydrophobic amino acids. You are provided with the following sequences: # Protein Sequence Sequence Sequence 1 MVLTIYPDELVQIVSDKIASNKLDVSVTPGKIMQLLRDNLTLWAAKILG Sequence 2

WQEGNVLHWTDKFTPLVQKAVAAEHLGKKVADALTNAVAHVDDMPNALVEE Sequence 3
GNWMLVFSLLMWLTAVFGYAYGAGVLGAGLAGLWAFSTQKDNSTTST Sequence 4
MQKVTLLVLLFLGAAFGSQDARQKSLNLEELKEKQKDFDQLKDF

For this task, the set of hydrophobic amino acids is defined as:

A, V, I, L, M, F, W, Y

Use custom functions (instead of internal functions) to perform the following tasks:

a)Write a function that calculates the length of each protein sequence. b) Write a function that calculates the hydrophobicity percentage of each protein sequence (i.e., the percentage of amino acids belonging to the hydrophobic set). c) Write a function that identifies which sequence has the highest hydrophobicity percentage. d) Write a function to compute the average hydrophobicity percentage across all protein sequences. e) Create a function that filters and returns all sequences with a hydrophobicity percentage greater than 45%.

The hydrophobic amino acid set utilised here, which includes A, V, I, L, M, F, W, and Y, is in line with commonly used hydrophobicity scales in the literature (Lehninger, Nelson, and Cox, 2017).

solution to Q2:

Firstly, i have to look at the Setup - Sequences and Hydrophobic Set

Storing the protein sequences with names for easy reference

```
sequences <- c(
  Seq1 = "MVLTIYPDELVQIVSDKIASNKLDVSVTPGKIMQLLRDNLTLWAAKILG",
  Seq2 = "WQEGNVLHWTDKFTPLVQKAVAAEHLGKKVADALTNAVAHVDDMPNALVEE",
  Seq3 = "GNWMLVFSLLMWLTAVFGYAYGAGVLGAGLAGLWAFSTQKDNSTTST",
  Seq4 = "MQKVTLLVLLFLGAAFGSQDARQKSLNLEELKEKQKDFDQLKDF"
)

# Hydrophobic amino acids as defined in the task
hydrophobic_aa <- c("A", "V", "I", "L", "M", "F", "W", "Y")
```

a) Function to calculates each Sequence Length

```
calculate_seq_length <- function(seq) {
  nchar(seq)
}

# Applying function to all sequences
seq_lengths <- sapply(sequences, calculate_seq_length)
seq_lengths
```

```
## Seq1 Seq2 Seq3 Seq4
##   49   51   47   44
```

b) Function to calculate Hydrophobicity Percentage of each protein sequence

```
calc_hydrophobicity <- function(seq, hydrophobic_set) {
  # Splitting the sequence into individual amino acids:
  amino_acids <- strsplit(seq, split = "")[[1]]
  # and in order to get the total number of amino acids
  total_aa    <- length(amino_acids)

  # to be able to count how many are in the hydrophobic set
  hydro_count <- sum(amino_acids %in% hydrophobic_set)

  # Calculate the hydrophobic set of count, and returning it as percentage
  (hydro_count / total_aa) * 100
}

# Applying the function to each by calculating for each sequence
hydro_percentages <- sapply(sequences, calc_hydrophobicity, hydrophobic_set = hydr
ophobic_aa)
round(hydro_percentages, 2)  # Rounding the results to 2 decimal places
```

```
##  Seq1  Seq2  Seq3  Seq4
## 51.02 47.06 55.32 43.18
```

c) Function to know the Sequence with Highest Hydrophobicity

```
find_most_hydrophobic <- function(seq_list, hydrophobic_set) {
  # to calculate hydrophobicity percentage for every sequence, i also need to make the result or output looks more simplified
  hydro_percent_values <- sapply(seq_list, calc_hydrophobicity, hydrophobic_set = hydrophobic_set)

  # Return or identify the name or sequence with the highest percentage
  most_hydrophobic_name <- names(which.max(hydro_percent_values))
}
# to determine which sequence is the most hydrophobic
most_hydrophobic <- find_most_hydrophobic(sequences, hydrophobic_aa)
# i can Concatenate and print the results
cat("Sequence with highest hydrophobicity:", most_hydrophobic, "\n")
```

```
## Sequence with highest hydrophobicity: Seq3
```

```
cat("Its hydrophobicity %:", round(hydro_percentages[most_hydrophobic], 2), "%\n")
```

```
## Its hydrophobicity %: 55.32 %
```

d) Function to calculate the average hydrophobicity across all sequences

```
calc_avg_hydrophobicity <- function(seq_list, hydrophobic_set) {
  # Computing hydrophobicity percentage for each sequence
  all_hydro_percentages <- sapply(seq_list, calc_hydrophobicity, hydrophobic_set = hydrophobic_set)
  # Return the average value of the %
  average_percentage <- mean(all_hydro_percentages)
}
# calculating the overall average hydrophobicity
avg_hydro <- calc_avg_hydrophobicity(sequences, hydrophobic_aa)
# Displaying or view the result
cat("Average hydrophobicity across all sequences:", round(avg_hydro, 2), "%\n")
```

```
## Average hydrophobicity across all sequences: 49.15 %
```

e) Function to select sequences with hydrophobicity greater than 45% (the given threshold).

```
filter_by_hydrophobicity <- function(seq_list, hydrophobic_set, threshold = 45) {
  # i) i need to calculate hydrophobicity % for each sequence
  hydro_percent_values <- sapply(seq_list, calc_hydrophobicity, hydrophobic_set =
hydrophobic_set)
  # ii) then, return only sequences greater than the chosen threshold, 45%.
  seq_list[hydro_percent_values > threshold]
}

# in my next line of code, Filtering sequences with hydrophobicity greater than 45
% will be done, therefore:
filtered_seqs <- filter_by_hydrophobicity(sequences, hydrophobic_aa, threshold =
45)
# show the names of the selected sequences
cat("Sequences with hydrophobicity > 45%:\n")
```

```
## Sequences with hydrophobicity > 45%:
```

```
print(names(filtered_seqs))
```

```
## [1] "Seq1" "Seq2" "Seq3"
```

```
# also show their hydrophobicity percentages as well
cat("\nHydrophobicity percentages of selected sequences:\n")
```

```
##
## Hydrophobicity percentages of selected sequences:
```

```
print(round(sapply(filtered_seqs, calc_hydrophobicity, hydrophobic_set = hydrophob
ic_aa), 2))
```

```
##  Seq1  Seq2  Seq3
## 51.02 47.06 55.32
```

```
## To offer clear, formatted headers throughout this my R assignment, I might tend to use cat() a lot.
# this is just to ensure my output(s) or final report is readable and well-structured
```

Q3 (10 marks) How would you create a new S4 class and its associated methods?

a)Create a class called Gene with slots for gene_name (character), length (numeric), and expression (numeric).

b)Instantiate an object of class Gene for a hypothetical gene (e.g., “BRCA1”, length 1863 bp, expression 10.5 TPM).

c)Define a method for this class to calculate and display the gene’s average expression per base (expression/length).

Solution to Q3

Firstly, i’m going to define an S4 class called “Gene” with three attributes(gene_name, length, and expression).

```
setClass("Gene",
  slots = list(
    gene_name  = "character",
    length     = "numeric",
    expression = "numeric"
  )
)
```

Proceeding to instantiate or create an instance of the Gene class

will be using BRCA1 as my example gene, with a length of 1863 bp and an expression value of 10.5 TPM.

```
brca1_gene <- new("Gene",
                  gene_name  = "BRCA1",
                  length     = 1863,
                  expression = 10.5)

# to check the above created object
brca1_gene
```

```
## An object of class "Gene"
## Slot "gene_name":
## [1] "BRCA1"
##
## Slot "length":
## [1] 1863
##
## Slot "expression":
## [1] 10.5
```

Next will be to define a method to compute average expression per base

define a generic function

```
setGeneric("avg_expression_per_base", function(gene_obj) {
  standardGeneric("avg_expression_per_base")
})
```

```
## [1] "avg_expression_per_base"
```

```
# next is to further define a method for objects of class "Gene"
setMethod("avg_expression_per_base", "Gene", function(gene_obj) {
   # Calculate expression per base
  avg_expression <- gene_obj@expression / gene_obj@length
   # print the details/summary of the Gene class
  cat("Gene:", gene_obj@gene_name, "\n")
  cat("Length:", gene_obj@length, "bp\n")
  cat("Expression:", gene_obj@expression, "TPM\n")
  cat("Average expression per base:", round(avg_expression, 6), "TPM/bp\n")

})

#  applying the method to the BRCA1 object
avg_expression_per_base(brca1_gene)
```

```
## Gene: BRCA1
## Length: 1863 bp
## Expression: 10.5 TPM
## Average expression per base: 0.005636 TPM/bp
```

Q4 (10 marks) A colony of algae can be modelled using a modified recurrence sequence. The colony starts with 3 algae cells, and each new generation is calculated using the rule:

New term = 3 × (sum of the previous two terms)

The sequence begins: 3, 9, 36, 135, …

a)Write a function that generates the first 20 terms of this algae growth sequence.

b)Write a function that determines how many of the first 20 terms are divisible by

9.

c) Write a function that calculates the total number of algae cells in the 20th generation if two cells die on each cycle.

SOLUTION to Q4

The sequence rule is: **New term = 3 × (sum of the previous two terms)**.
Starting values: 3, 9. So term 3 = 3×(3+9) = 36, term 4 = 3×(9+36) = 135, and so on.

a) first sub-task of Q4 is to create a function that generate the algae growth sequence .i.e. to generate the First 20 Terms

```
generate_algae_seq <- function(n = 20) {
  # i want to set up a numeric vector to store the sequence
  seq_values <- numeric(n)  # n = number of terms
   # next is to set the first two starting values
  seq_values[1] <- 3
  seq_values[2] <- 9
  # will further generate the remaining terms using the given rule
  for (i in 3:n) {
    seq_values[i] <- 3 * (seq_values[i - 2] + seq_values[i - 1])
  }
  # to return all generated terms:
  seq_values
}
# in order to generate and show the first 20 terms
algae_seq <- generate_algae_seq(20)
cat("First 20 terms of the algae sequence:\n")
```

```
## First 20 terms of the algae sequence:
```

```
print(algae_seq)
```

```
##  [1]            3            9           36          135          513
##  [6]         1944         7371        27945       105948       401679
## [11]      1522881      5773680     21889683     82990089    314639316
## [16]   1192888215   4522582593  17146412424  65006985051 246460192425
```

b) the next sub-task requires me to do a count of how many terms are divisible by 9

```
# therefore, function to count how many values are divisible by 9
count_divisible_by_9 <- function(seq_values) {
  count <- 0
 # Checking each value's divisibility by iterating through it.
  for (value in seq_values) {
    if (value %% 9 == 0) {
      count <- count + 1
    }
  }
  count
}
# to count or determine how many of the first 20 terms are divisible by 9
divisible_by_9_count <- count_divisible_by_9(algae_seq)
cat("Number of the first 20 terms divisible by 9:", divisible_by_9_count, "\n")
```

```
## Number of the first 20 terms divisible by 9: 19
```

```
# to show the actual values that are divisible by 9
cat("Terms divisible by 9:\n")
```

```
## Terms divisible by 9:
```

```
print(algae_seq[algae_seq %% 9 == 0])
```

```
##  [1]            9           36          135          513         1944
##  [6]         7371        27945       105948       401679      1522881
## [11]      5773680     21889683     82990089    314639316   1192888215
## [16]   4522582593  17146412424  65006985051 246460192425
```

c) Next or last sub-tasks for Q4 is to calculate the total Cells in the 20th Generation with 2 Deaths Per Cycle

Since two cells each cycle die, the actual count at generation 20 is: the raw 20th term minus two deaths multiplied by the number of cycles that have passed.

```
# Function to calculate the adjusted population at the 20th generation will be:
calc_adjusted_20th_gen <- function(seq_values, deaths_per_cycle = 2) {
  n <- 20  # n here = generation number
  # to get the raw count or value at generation 20
  raw_count   <- seq_values[n]
  total_deaths <- deaths_per_cycle * n
  # Adjusting the final value
  adjusted_value    <- raw_count - total_deaths

  cat("Raw 20th generation count:", raw_count, "\n")
  cat("Total deaths over 20 cycles:", total_deaths, "\n")
  cat("Adjusted value at 20th generation:", adjusted_value, "\n")
  # to return the adjusted value
  adjusted_value
}

calc_adjusted_20th_gen(algae_seq)
```

```
## Raw 20th generation count: 246460192425
## Total deaths over 20 cycles: 40
## Adjusted value at 20th generation: 246460192385
```

```
## [1] 246460192385
```

Q5 (40 marks) Tables GSE9476_series_matrix.txt (tab-delimited) and GSE9476_meta_info.txt are the microarray gene expression dataset and sample information table about patients with Acute Myeloid Leukemia (AML) and normal subjects as Controls.

The expression data have been log2 transformed. Please complete the following tasks.

1.How many patients are with AML and

how many are the control cases? How many genes(probesets) are we studying?

(For the purpose of the assignment, each probeset is treated as a gene). # 2.Create your R function to calculate the average age of AML patients and Control subjects separately. Check your results by using R mean function. # 3.Is there any significant difference between sample ages of AMLs and Controls? Assume the age follows a normal distribution. # 4.Make a boxplot of gene expression for all the samples, marking data points in red for AML samples and blue for Control samples. # 5.Create a table to include mean_AML, sd_AML, min_AML, max_AML, mean_Control, sd_Control, min_Control, max_Control, and Fold_change (i.e, mean_AML – mean_Control) for each gene. It is fine to use built-in functions. Hints: split the dataset into AML data and normal data sets, and then for each gene/probeset, calculate its mean, standard derivation, min, and max expression values across samples separately (using a built-in function), and further merge these statistical values for each gene into one table. Apply data.frame() and give the created table the same row names as the gene expression data table. # 6.With the table created above, make a scatterplot of the mean expression of AMLvs. Control samples. Are mean_AML and mean_Control significantly different? # 7.For each gene/probeset, conduct t-test between AML samples and Control samples, and then create a table including t scores and p-values from t-test, and merge to the table created in 6. # 8.How many genes/probesets are left after the following filters? a.) mean_AML > 3 and mean_Control > 3 b.) max_AML < 14 or max_Control < 14 c)Apply a) and b) filters together which is more stringent than applying a) or b) alone # 9.Following 8, if we further filter gene/probeset by applying p-value < 0.05 and the absolute value of log2 Fold_change bigger than 1, then how many genes will be left which we call them significantly expressed genes (DEGs)? Among them, how many are up-regulated (Fold_change > 0) and how many are down-regulated (Fold_change < 0)? List top 10 most upregulated DEGs and 10 most down-regulated DEGs including their p-value, log2FC, and mean values in each sample group. # 10.If you apply the limma package rather than t-test, then how many DEGs will be discovered by limma? (p-value < 0.05 & abs(log2FC) > 1). How many overlapped DEGs by these two different approaches? please draw a Venn diagram for demonstration.

SOLUTION TO Q5 (q5.01 - q5.10)

AML Microarray Gene Expression Analysis

```
# firstly, i need to re-load my required libraries for plotting and analysis
library(ggplot2)
library(limma)
library(VennDiagram)
```

```
## Loading required package: grid
```

```
## Loading required package: futile.logger
```

```
library(grid)
```

then, i can proceed to load the Data

the header section of the series matrix file (lines beginning with `!`) has been excluded. The data expression comes after `!series_matrix_table_begin`.

```
# Loading of gene expression data by skipping the GEO file's metadata lines.
## Read.table() is used here with comment.char = "!" to automatically skip the hea
der lines that begin with "!" in the GSE series matrix file, before reading the ex
pression data.
expression_data <- read.table(
  "GSE9476_series_matrix.txt",
  comment.char   = "!",
  header         = TRUE,
  sep            = "\t",
  row.names      = 1,
  check.names    = FALSE
)

# have to load sample metadata, which consist of age and disease status.
metadata <- read.table(
  "GSE9476_meta_info.txt",
  header    = TRUE,
  sep       = "\t",
  row.names = 1
)

# Quick one: Checking dimensions to confirm successful loading of both data
dim(expression_data)
```

```
## [1] 22283    64
```

```
dim(metadata)
```

```
## [1] 64  3
```

Q5.01 - which requires me to check how many patients are of AML, Control cases and the Probesets(genes) that we're studying ?

```
num_aml     <- sum(metadata$Disease_status == "AML")
num_control <- sum(metadata$Disease_status == "Control")
num_genes  <- nrow(expression_data)

cat("Number of AML samples:     ", num_aml, "\n")
```

```
## Number of AML samples:      26
```

```
cat("Number of Control samples: ", num_control, "\n")
```

```
## Number of Control samples:  38
```

```
cat("Number of probesets (genes):", num_genes, "\n")
```

```
## Number of probesets (genes): 22283
```

Q5.02 - Average Age of AML and Control Samples (Custom Function + built-in check to verify)

```
# # my custom function to calculate mean (without using built-in mean())
my_mean <- function(x) {
  x_clean <- x[!is.na(x)]   # removing missing values (NAs) manually
  total   <- 0
  for (val in x_clean) {
    total <- total + val
  }
  # to return (my custom function) average
  total / length(x_clean)
}
# for me to extract ages by group
aml_ages     <- metadata$Age[metadata$Disease_status == "AML"]
control_ages <- metadata$Age[metadata$Disease_status == "Control"]
# i have to compute means using custom function
custom_mean_aml     <- my_mean(aml_ages)
custom_mean_control <- my_mean(control_ages)

cat("Custom function - Mean age AML:     ", round(custom_mean_aml, 2), "\n")
```

```
## Custom function - Mean age AML:      56.12
```

```
cat("Custom function - Mean age Control: ", round(custom_mean_control, 2), "\n")
```

```
## Custom function - Mean age Control:  33.33
```

```
# i can further verify my custom function mean value, by using R's built-in functi
on, mean(), in the form of :
builtin_mean_aml     <- mean(aml_ages, na.rm = TRUE)
builtin_mean_control <- mean(control_ages, na.rm = TRUE)

cat("\nBuilt-in mean() - Mean age AML:     ", round(builtin_mean_aml, 2), "\n")
```

```
##
## Built-in mean() - Mean age AML:      56.12
```

```
cat("Built-in mean() - Mean age Control: ", round(builtin_mean_control, 2), "\n")
```

```
## Built-in mean() - Mean age Control:  33.33
```

Q5.3 - Is There a Significant Age Difference Between AML and Controls samples?

Since we are instructed to assume normality, I will apply a two-sample t-test.

```
# for me to perform two-sample t-test, then:
age_t_test <- t.test(aml_ages, control_ages, na.action = na.omit)
# to view the whole test results
age_t_test
```

```
##
##  Welch Two Sample t-test
##
## data:  aml_ages and control_ages
## t = 6.2687, df = 47.986, p-value = 9.743e-08
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
##  15.47480 30.08931
## sample estimates:
## mean of x mean of y
##  56.11538  33.33333
```

```
# interpretation of the result leading to its conclusion
if (age_t_test$p.value < 0.05) {
  cat("\nConclusion: There IS a statistically significant difference in age",
      "between AML patients and Controls (p =", round(age_t_test$p.value, 4), ").\
n")
} else {
  cat("\nConclusion: There is NO statistically significant difference in age",
      "between AML patients and Controls (p =", round(age_t_test$p.value, 4), ").\
n")
}
```

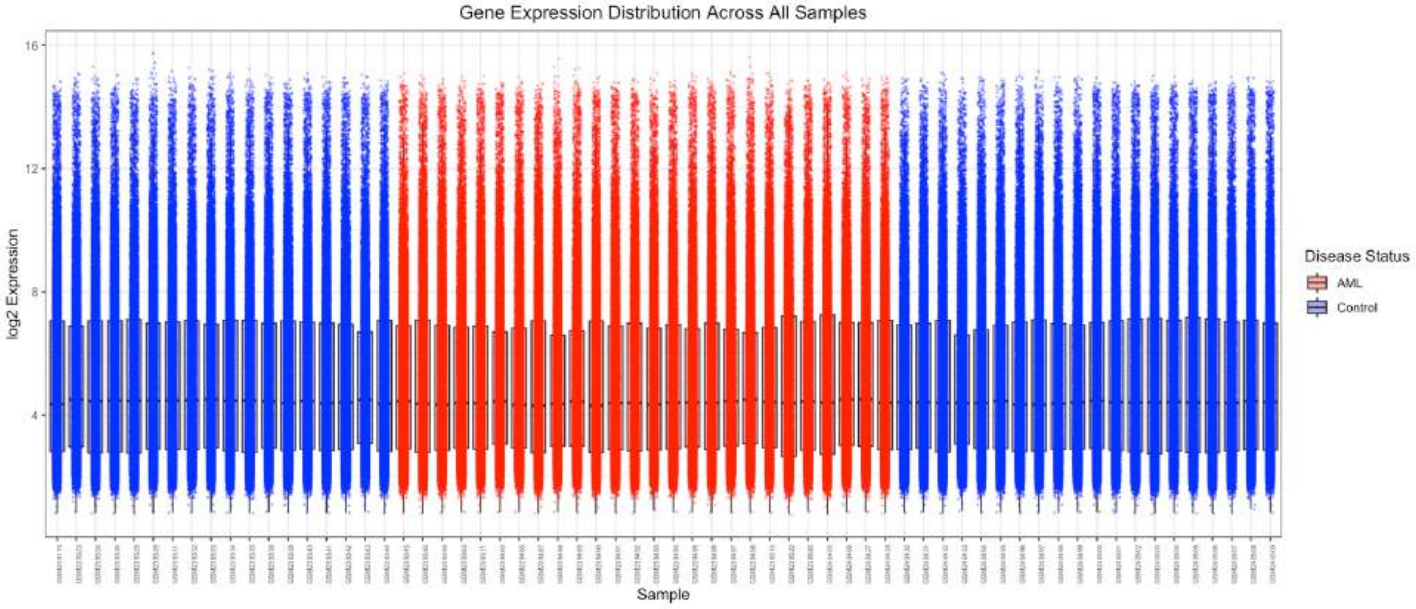
```
##
## Conclusion: There IS a statistically significant difference in age between AML
patients and Controls (p = 0 ).
```

Q5.4 - requires me to make a boxplot of Gene Expression for All Samples

The AML sample data points are coloured **red**, while the Control data points are **blue**.

```
# While fig.height protects my constructed boxplots from seeming overly
# stretched out, fig.width helps stretch my plot to suit the several
# samples side by side.

# using ggplot2 as taught in Week 8 for data visualisation

# to identify which samples are AML vs Control
aml_sample_ids     <- rownames(metadata)[metadata$Disease_status == "AML"]
control_sample_ids <- rownames(metadata)[metadata$Disease_status == "Control"]

# Collecting all sample IDs from the expression data
all_sample_ids <- colnames(expression_data)

# Converting expression data to long format so each row
# represents one gene expression value for one sample
long_format <- do.call(rbind, lapply(all_sample_ids, function(sample_id) {
  data.frame(
    Sample     = sample_id,
    Expression = expression_data[[sample_id]],
    Group      = ifelse(sample_id %in% aml_sample_ids, "AML", "Control"),
    stringsAsFactors = FALSE
  )
}))

# Building the boxplot using ggplot2 layers as taught in Week 8.
# geom_boxplot() provides me the distribution summary for each sample.
# geom_jitter() overlays individual data points on top of each box,
```

```
# with red points for AML samples and blue points for Control samples (as instruct
ed in the task).
ggplot(long_format, aes(x = Sample, y = Expression, fill = Group)) +
  geom_boxplot(outlier.shape = NA,
               alpha         = 0.4) +
  geom_jitter(aes(colour = Group),
              width = 0.2,
              size  = 0.3,
              alpha = 0.3) +
  scale_colour_manual(values = c("AML"     = "red",
                                 "Control" = "blue")) +
  scale_fill_manual(values   = c("AML"     = "red",
                                 "Control" = "blue")) +
  labs(
    title  = "Gene Expression Distribution Across All Samples",
    x      = "Sample",
    y      = "log2 Expression",
    colour = "Disease Status",
    fill   = "Disease Status"
  ) +
  theme_bw() +
  theme(
    axis.text.x = element_text(angle = 90,
                               hjust = 1,
                               size  = 5),
    plot.title  = element_text(hjust = 0.5)
  )
```



Q5.5 - requires me to create a table of Summary Statistics for each Gene

```
# split expression data by group
aml_expr_data     <- expression_data[, aml_sample_ids]
control_expr_data <- expression_data[, control_sample_ids]

# Using apply to calculate statistics for each gene.
mean_AML     <- apply(aml_expr_data,     1, mean)
sd_AML       <- apply(aml_expr_data,     1, sd)
min_AML      <- apply(aml_expr_data,     1, min)
max_AML      <- apply(aml_expr_data,     1, max)

mean_Control <- apply(control_expr_data, 1, mean)
sd_Control   <- apply(control_expr_data, 1, sd)
min_Control  <- apply(control_expr_data, 1, min)
max_Control  <- apply(control_expr_data, 1, max)

# Fold change = mean_AML – mean_Control (log2 scale, so this is log2FC)
Fold_change  <- mean_AML - mean_Control

# Combining into one table or data frame
stats_table <- data.frame(
  mean_AML,
  sd_AML,
  min_AML,
  max_AML,
  mean_Control,
  sd_Control,
  min_Control,
  max_Control,
  Fold_change
)
rownames(stats_table) <- rownames(expression_data)

# to preview first few or 6 rows of my table
head(stats_table)
```

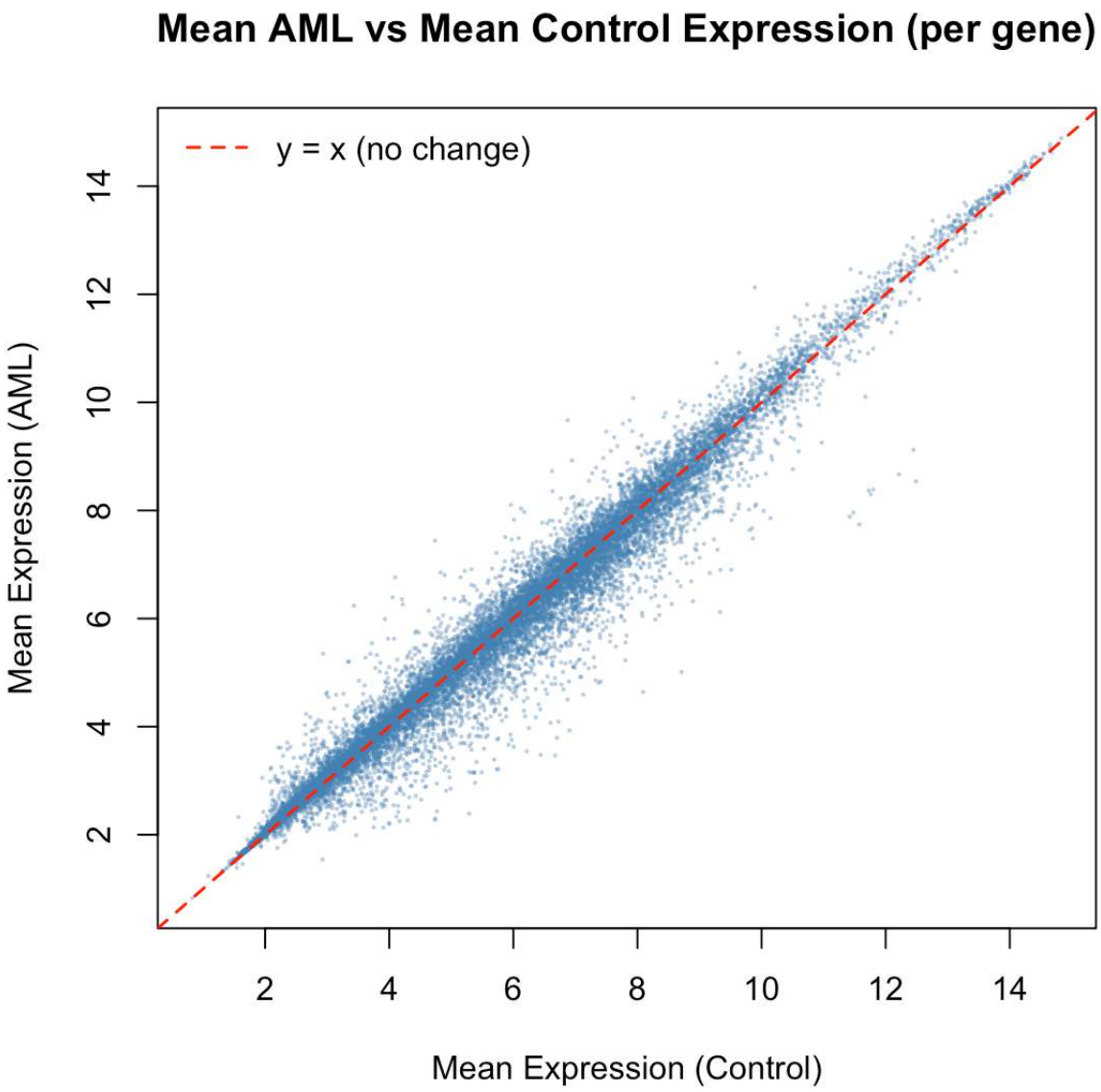
```
##           mean_AML     sd_AML  min_AML  max_AML mean_Control sd_Control
## 1007_s_at 3.190831 0.28688658 2.738629 4.189943     3.109452 0.13263540
## 1053_at   6.497534 0.43710532 5.707238 7.603169     6.671247 0.54927460
## 117_at    5.352038 1.54329303 3.961892 9.644181     6.475112 2.63384079
## 121_at    7.449728 0.35013512 6.665644 8.221074     7.176393 0.26198848
## 1255_g_at 2.265448 0.07559105 2.068680 2.398069     2.239653 0.07173247
## 1294_at   7.870432 0.63481967 6.757480 9.531410     7.715736 0.52599571
##           min_Control max_Control Fold_change
## 1007_s_at    2.843774    3.448303  0.08137906
## 1053_at      5.932384    8.143075 -0.17371329
## 117_at       3.923634   12.358038 -1.12307397
## 121_at       6.771577    8.019825  0.27333450
## 1255_g_at    2.099221    2.512393  0.02579531
## 1294_at      6.706907    8.929810  0.15469537
```

```
cat("Dimensions of stats table:", dim(stats_table), "\n")
```

```
## Dimensions of stats table: 22283 9
```

Q5.6 - Scatterplot of mean_AML vs mean_Control

```
plot(stats_table$mean_Control, stats_table$mean_AML,
     pch  = 16,
     cex  = 0.3,
     col  = adjustcolor("steelblue", alpha.f = 0.4),
     xlab = "Mean Expression (Control)",
     ylab = "Mean Expression (AML)",
     main = "Mean AML vs Mean Control Expression (per gene)")

# in my next line of code, the reference line remain unchanged
abline(a = 0, b = 1, col = "red", lty = 2, lwd = 1.5)
legend("topleft", legend = "y = x (no change)", col = "red",
       lty = 2, lwd = 1.5, bty = "n")
```



Interpretation: Given that most genes are not differentially expressed, a strong positive correlation between mean AML and mean Control expression is evident. A subset of probesets is differentially expressed between the two groups, as evidenced by numerous points that fall above or below the identity line (y = x).

I can also perform a paired t-test across all probesets to directly determine whether mean_AML and mean_Control are significantly different overall:

```
# Note: this is included to check whether there is a significant overall differenc
e when using a paired t-test.
# like i said in line 683, i want to try paired t-test to compare the overall mean
AML vs Control (n=paired samples):
overall_test <- t.test(stats_table$mean_AML, stats_table$mean_Control, paired = TR
UE)
overall_test
```

```
##
##  Paired t-test
##
## data:  stats_table$mean_AML and stats_table$mean_Control
## t = -6.8546, df = 22282, p-value = 7.336e-12
## alternative hypothesis: true mean difference is not equal to 0
## 95 percent confidence interval:
##  -0.02303778 -0.01279226
## sample estimates:
## mean difference
##     -0.01791502
```

```
# verification of statistical signifcance via the p-value
cat("\nOverall paired t-test p-value:", format(overall_test$p.value, scientific =
TRUE), "\n")
```

```
##
## Overall paired t-test p-value: 7.335785e-12
```

Q5.7 - Per-Gene t-test and Merged Table

```
# function to run a t-test for each gene between AML and Control samples
t_test_per_gene <- function(expr_data, aml_ids, control_ids) {
  results <- t(apply(expr_data, 1, function(row) {
    aml_vals  <- as.numeric(row[aml_ids])
    control_vals <- as.numeric(row[control_ids])
    test        <- t.test(aml_vals, control_vals)
    c(t_score = as.numeric(test$statistic),
      p_value = test$p.value)
  }))
  as.data.frame(results)
}

t_test_results <- t_test_per_gene(expression_data, aml_sample_ids, control_sample_
ids)
rownames(t_test_results) <- rownames(expression_data)
head(t_test_results)
```

```
##             t_score     p_value
## 1007_s_at  1.350983 0.186066483
## 1053_at   -1.404938 0.165154359
## 117_at    -2.144890 0.035966515
## 121_at     3.384720 0.001518826
## 1255_g_at  1.368728 0.176971972
## 1294_at    1.024930 0.310643245
```

```
# Next is to Merge t-test results into the main stats table
stats_table <- cbind(stats_table, t_test_results)   # cbind() helps me join  the s
eperate columns into a single table (just as i want for the merging)
cat("Stats table now has columns:", ncol(stats_table), "\n")
```

```
## Stats table now has columns: 11
```

```
cat("Column names:", paste(names(stats_table), collapse = ", "), "\n")
```

```
## Column names: mean_AML, sd_AML, min_AML, max_AML, mean_Control, sd_Control, min
_Control, max_Control, Fold_change, t_score, p_value
```

Q5.8 – Filtering the Gene Table, and looking at the number of genes left after that

for the Filter a) mean_AML > 3 and mean_Control > 3

```
filter_a <- stats_table[stats_table$mean_AML > 3 & stats_table$mean_Control > 3, ]
cat("Genes remaining after filter a) (mean_AML > 3 and mean_Control > 3):",
    nrow(filter_a), "\n")
```

```
## Genes remaining after filter a) (mean_AML > 3 and mean_Control > 3): 16026
```

while for the Filter b) max_AML < 14 or max_Control < 14

```
filter_b <- stats_table[stats_table$max_AML < 14 | stats_table$max_Control < 14, ]
cat("Genes remaining after filter b) (max_AML < 14 or max_Control < 14):",
    nrow(filter_b), "\n")
```

```
## Genes remaining after filter b) (max_AML < 14 or max_Control < 14): 22171
```

lastly for Filter c): will have to Apply Both a) and b) Together

```
# Combining both filters together, .ie. filter_a_and_b
filter_a_and_b <- stats_table[
  (stats_table$mean_AML > 3 & stats_table$mean_Control > 3) &
  (stats_table$max_AML < 14 | stats_table$max_Control < 14),
]
cat("Genes remaining after applying BOTH filters a) and b):",
    nrow(filter_a_and_b), "\n")
```

```
## Genes remaining after applying BOTH filters a) and b): 15914
```

```
# Verifying the logic as also stipulated in the question 5.8c: The decrease in the number of genes validates the stricter combined filters criteria.
cat("\nAs I expected, applying both filters together is more stringent than",
    "applying filter a or b alone, resulting in fewer genes.\n")
```

```
##
## As I expected, applying both filters together is more stringent than applying f
ilter a or b alone, resulting in fewer genes.
```

Q5.9 - to apply the DEG criteria: p-value < 0.05 and |log2FC| > 1,

then:

```
# Applying DEG criteria, and Starting from the a+b filtered set (filter_a_and_b fr
om Q5.8)
degs <- filter_a_and_b[
  filter_a_and_b$p_value < 0.05 &
  abs(filter_a_and_b$Fold_change) > 1,
]

# next would be to Split into up/down regulated
upregulated   <- degs[degs$Fold_change > 0, ]
downregulated <- degs[degs$Fold_change < 0, ]

cat("Total DEGs:", nrow(degs), "\n")
```

```
## Total DEGs: 626
```

```
cat("Upregulated:", nrow(upregulated), "\n")
```

```
## Upregulated: 136
```

```
cat("Downregulated:", nrow(downregulated), "\n")
```

```
## Downregulated: 490
```

Below is the line of codes for the List of top 10 Upregulated DEGs

```
top10_up <- head(upregulated[order(-upregulated$Fold_change),
                          c("mean_AML", "mean_Control", "Fold_change", "p_value")]
, 10)
knitr::kable(round(top10_up, 4),
             caption = "Top 10 Most Upregulated DEGs (Highest log2FC)")
```

Top 10 Most Upregulated DEGs (Highest log2FC)

	mean_AML	mean_Control	Fold_change	p_value
215489_x_at	6.2375	3.4312	2.8064	0.0000
206674_at	9.6698	6.8764	2.7934	0.0000
210365_at	7.4411	4.7369	2.7042	0.0000
204647_at	6.7641	4.0967	2.6674	0.0000
206067_s_at	6.3920	4.0543	2.3376	0.0000
201105_at	12.1275	9.8913	2.2362	0.0000
205131_x_at	6.3546	4.2041	2.1505	0.0008
214575_s_at	8.1073	5.9600	2.1474	0.0064
205382_s_at	10.0785	7.9316	2.1469	0.0008
211709_s_at	8.7317	6.6246	2.1071	0.0003

and the list of top 10 Downregulated DEGs is:

```
top10_down <- head(downregulated[order(downregulated$Fold_change),
                              c("mean_AML", "mean_Control", "Fold_change", "p_valu
e")], 10)
knitr::kable(round(top10_down, 4),
             caption = "Top 10 Most Down-Regulated DEGs (lowest log2FC)")
```

Top 10 Most Down-Regulated DEGs (lowest log2FC)

	mean_AML	mean_Control	Fold_change	p_value
209116_x_at	8.5385	12.4893	-3.9509	0e+00
217414_x_at	7.7409	11.5781	-3.8372	0e+00
212224_at	5.0108	8.7108	-3.7000	0e+00

217232_x_at	8.6683	12.2133	-3.5451	0e+00
209458_x_at	7.8844	11.4194	-3.5350	0e+00
211699_x_at	7.9627	11.4796	-3.5170	0e+00
204018_x_at	8.3000	11.7588	-3.4589	0e+00
206207_at	4.6386	8.0910	-3.4523	0e+00
211745_x_at	8.3879	11.8028	-3.4149	0e+00
214414_x_at	8.3704	11.7267	-3.3563	1e-04

```
#  as seen in the rmarkdown sheet, by using the caption argument in both lists (up
-and-down). i am giving the tables a formal title and making sure it is correctly
placed in the finished document.
```

Q5.10 - limma Analysis and Venn Diagram Comparison (between the overlapping DEGs of limma and t-test)

By using a linear modelling framework with empirical Bayes moderation of variance estimates, the limma package provides a more statistically robust alternative to the t-test approach used in Q5.7, which treats each gene independently. This increases statistical power, especially when sample sizes are small (Smyth, 2004).

```
# to apply the limma package, i need to build the design matrix for limma
group_labels <- ifelse(colnames(expression_data) %in% aml_sample_ids, "AML", "Cont
rol")
group_factor <- factor(group_labels, levels = c("Control", "AML"))
design_matrix   <- model.matrix(~ group_factor)

# Fit the linear model
fit1 <- lmFit(as.matrix(expression_data), design_matrix)
fit2 <- eBayes(fit1)

# Extraction of results for all probesets/genes
limma_results <- topTable(fit2, number = Inf, sort.by = "none")
```

```
## Removing intercept from test coefficients
```

```
head(limma_results)
```

```
##                logFC  AveExpr         t     P.Value  adj.P.Val          B
## 1007_s_at  0.08137906 3.142512  1.528361 0.131409557 0.28609968 -5.2694322
## 1053_at   -0.17371329 6.600676 -1.356992 0.179607656 0.34012370 -5.5057888
## 117_at    -1.12307397 6.018863 -1.973170 0.052850171 0.17480486 -4.5367279
## 121_at     0.27333450 7.287435  3.588617 0.000648726 0.01056693 -0.6246912
## 1255_g_at  0.02579531 2.250132  1.286268 0.203038622 0.36530558 -5.5956289
## 1294_at    0.15469537 7.778581  1.070925 0.288273248 0.45133590 -5.8409232
```

```
# Using the same pre-filtering (filter_a_and_b row names) before calling in DEGs
# limma analysis will be performed on the filtered data, since we are doing a comp
arison with t-test (on the remaining pool of genes)

filtered_limma <- limma_results[rownames(filter_a_and_b), ]

# Identify limma DEGs with same thresholds
limma_degs <- filtered_limma[
  filtered_limma$P.Value < 0.05 &
  abs(filtered_limma$logFC) > 1,
]
cat("Number of DEGs found by limma:", nrow(limma_degs), "\n")
```

```
## Number of DEGs found by limma: 626
```

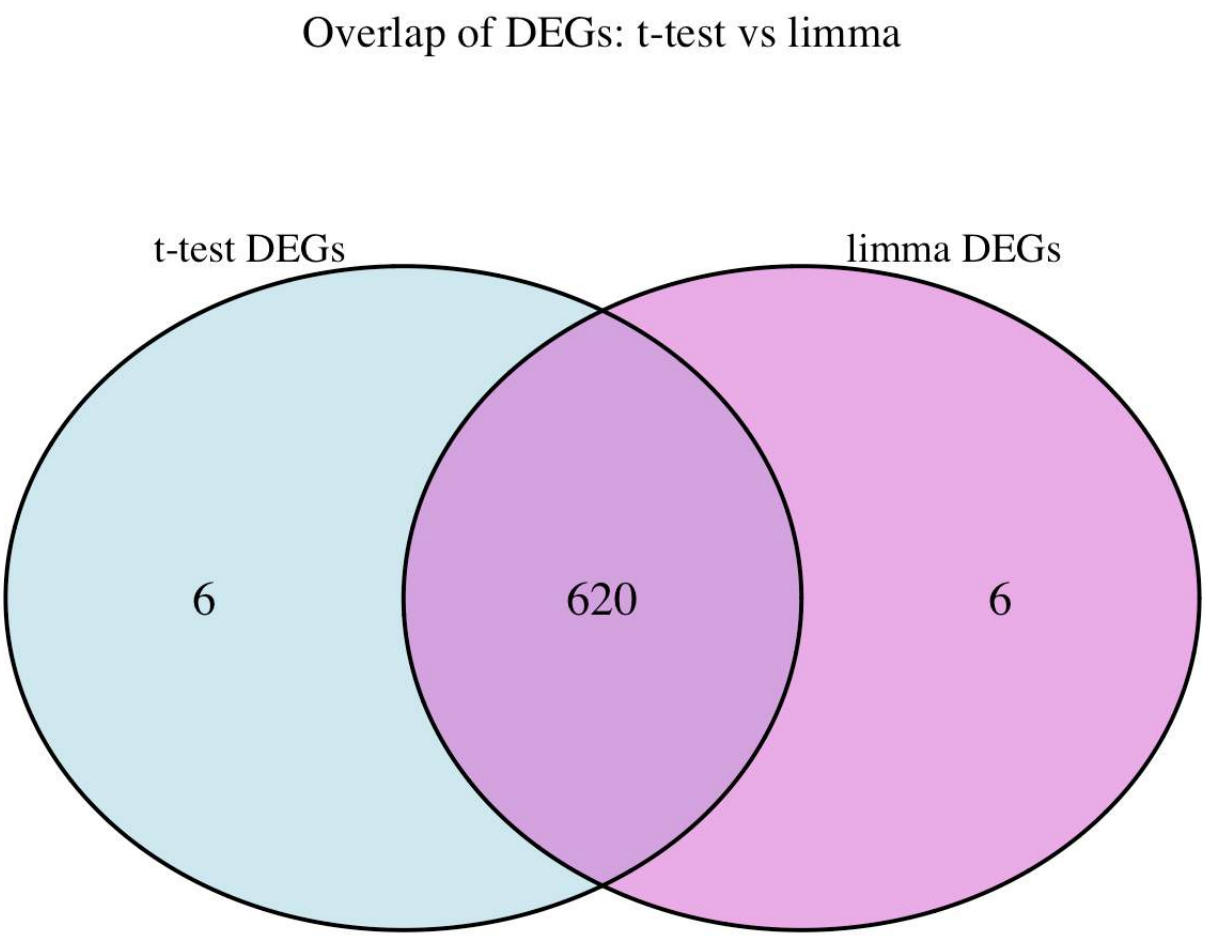
```
# looking at the DEG gene lists, and to check the overlaps
t_test_deg_ids <- rownames(degs)
limma_deg_ids <- rownames(limma_degs)

limma_unique <- setdiff(limma_deg_ids, t_test_deg_ids)
print(paste("Genes unique to limma:", length(limma_unique)))
```

```
## [1] "Genes unique to limma: 6"
```

```
overlap_ids <- intersect(t_test_deg_ids, limma_deg_ids)
cat("Overlapping DEGs between t-test and limma:", length(overlap_ids), "\n")
```

```
## Overlapping DEGs between t-test and limma: 620
```

```
# as the third task of Q5.10 stipulated, it stated to draw Venn diagram for demons
traion of the overlapped DEGs.
# therefore, creating a Venn diagram to visualize overlap between DEGs identified
by the t-test and limma methods
venn_plot <- venn.diagram(
  x = list(
    "t-test DEGs" = t_test_deg_ids,
    "limma DEGs"  = limma_deg_ids
  ),
  filename        = NULL,
  fill            = c("lightblue", "orchid"),
  alpha           = 0.6,
  cex             = 1.4,
  cat.cex         = 1.2,
  cat.pos         = c(-20, 20),
  scaled          = FALSE,
  euler.d.        = FALSE,
  main            = "Overlap of DEGs: t-test vs limma",
  main.cex        = 1.3
)

# to draw the Venn diagram
grid.newpage()
grid.draw(venn_plot)
```




```
# Print a clean summary of the overlaps
cat("\nSummary of DEG overlap:\n")
```

```
##
## Summary of DEG overlap:
```

```
cat("DEGs identified only by t-test:", length(setdiff(t_test_deg_ids, limma_deg_id
s)), "\n")
```

```
## DEGs identified only by t-test: 6
```

```
cat("DEGs identified only by limma: ", length(setdiff(limma_deg_ids, t_test_deg_id
s)), "\n")
```

```
## DEGs identified only by limma:  6
```

```
cat("DEGs identified by both:       ", length(overlap_ids), "\n")
```

```
## DEGs identified by both:        620
```

Interpretation: For microarray data, limma uses an empirical Bayes stabilization of variance estimates across genes, which is typically more potent and reliable, especially when sample sizes are small (such as the 26 AML and 38 Control samples in this example). The degree of agreement between the two methods is shown in the Venn diagram and The most reliable options for further biological research are usually DEGs that overlap between limma and the t-test.

REFERENCES

Chambers, J.M. (2008) Software for Data Analysis: Programming with R. New York: Springer.

Eddelbuettel, D. and François, R. (2011) 'Rcpp: seamless R and C++ integration', Journal of Statistical Software, 40(8), pp. 1-18. Available at: https://doi.org/10.18637/jss.v040.i08 (https://doi.org/10.18637/jss.v040.i08)

Gentleman, R.C. et al. (2004) 'Bioconductor: open software development for computational biology and bioinformatics', Genome Biology, 5(10), p. R80. Available at: https://doi.org/10.1186/gb-2004-5-10-r80 (https://doi.org/10.1186/gb-2004-5-10-r80)

Gillespie, C. and Lovelace, R. (2017) Efficient R Programming: A Practical Guide to Smarter Programming. 1st edn. Sebastopol: O'Reilly Media.

Ihaka, R. and Gentleman, R. (1996) 'R: a language for data analysis and graphics', Journal of Computational and Graphical Statistics, 5(3), pp. 299-314.

R Core Team (2024) R: A Language and Environment for Statistical Computing. Vienna: R Foundation for Statistical Computing. Available at: https://www.R-project.org (https://www.R-project.org) (Accessed: 5 May 2026).

Smyth, G.K. (2004) 'Linear models and empirical Bayes methods for assessing differential expression in microarray experiments', Statistical Applications in Genetics and Molecular Biology, 3(1), Article 3.
Wickham, H. and Grolemund, G. (2017) R for Data Science: Import, Tidy, Transform, Visualize, and Model Data. Sebastopol: O'Reilly Media.

Session Information

```
sessionInfo()
```

```
## R version 4.3.3 (2024-02-29)
## Platform: x86_64-apple-darwin20 (64-bit)
## Running under: macOS Big Sur 11.7.11
##
## Matrix products: default
## BLAS:   /Library/Frameworks/R.framework/Versions/4.3-x86_64/Resources/lib/libRb
las.0.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.3-x86_64/Resources/lib/libRl
apack.dylib;  LAPACK version 3.11.0
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: Europe/London
## tzcode source: internal
##
## attached base packages:
## [1] grid      stats     graphics  grDevices utils     datasets  methods
## [8] base
##
## other attached packages:
## [1] VennDiagram_1.7.3   futile.logger_1.4.9 limma_3.58.1
## [4] ggplot2_3.5.2
##
## loaded via a namespace (and not attached):
##  [1] gtable_0.3.6         jsonlite_2.0.0       dplyr_1.1.4
##  [4] compiler_4.3.3       tidyselect_1.2.1     jquerylib_0.1.4
##  [7] scales_1.4.0         yaml_2.3.12          fastmap_1.2.0
## [10] statmod_1.5.0        R6_2.6.1             labeling_0.4.3
## [13] generics_0.1.4       knitr_1.51           tibble_3.3.1
## [16] bslib_0.10.0         pillar_1.11.1        RColorBrewer_1.1-3
## [19] rlang_1.2.0          cachem_1.1.0         xfun_0.57
## [22] sass_0.4.10          otel_0.2.0           cli_3.6.6
## [25] withr_3.0.2          magrittr_2.0.5       formatR_1.14
## [28] futile.options_1.0.1 digest_0.6.39        rstudioapi_0.18.0
## [31] lifecycle_1.0.5      vctrs_0.7.3          evaluate_1.0.5
## [34] glue_1.8.1           farver_2.1.2         lambda.r_1.2.4
## [37] rmarkdown_2.31       tools_4.3.3          pkgconfig_2.0.3
## [40] htmltools_0.5.9
```